androsec

EXPLOITATION REPORT

Possible card number

re.pwnme

Package re.pwnme

Severity High

Generated 2026-11-12 18:23 UTC

Disclaimer

This report is an automated assessment and provides an approximate security overview. It is not guaranteed to be 100% accurate. Some vulnerabilities may not have been detected.

Assessment target

Application and device used for this verification.

App name	re.pwnme
Package	re.pwnme
Version	1.0
Min / Target SDK	21 / 29
APK size	3.5 MB
SHA-256	73799f94d9bce055509a14b1d33297aea81177d6a8ab8278dabfde3c17d8b41a
Device	VirtualBox
Android	9
Serial	10.0.2.5:5555
Report ID	exploitation_20261112_182234
Generated	2026-11-12 18:23 UTC

Executive summary

Finding severity **High** — app risk **MEDIUM** — score **31/100**

0	2	3	0	6
Critical	High	Medium	Low	Info

High Possible card number

RULE	LOG_CREDIT_CARD
CONFIDENCE	Medium
MASVS	MASVS-STORAGE-2
EVIDENCE	`F/DEBUG ( 2933): #00 pc 0000000000044951 /data/app/re.pwnme-0shNWy7MQKRBOJ7li5D7NA==/lib/x86_64/libnative-lib.so`
RECOMMENDATION	Never log PANs.

Assessment write-up

Narrative produced for this finding, including scope, methodology, evidence, and remediation.

Executive Summary

This report focuses on the specific finding **LOG_CREDIT_CARD — Possible card number**, rated **High** severity with **Medium** confidence.

Static analysis identified a possible card number logging path in the native library `libnative-lib.so` for the target application `re.pwnme`. The static evidence provided is a `F/DEBUG` backtrace line referencing:

`/data/app/re.pwnme-0shNWy7MQKRBOJ7li5D7NA==/lib/x86_64/libnative-lib.so`

Dynamic verification was attempted. The application was launched successfully, but the logcat collection steps failed due to an invalid command option. No live card number was captured from logcat during this run. Therefore, the report confirms the static finding, but live dynamic confirmation of exposed PAN data was not obtained.

If a PAN is written to Android system logs, the business impact is serious: cardholder data may be exposed to any local user or process with access to logcat, potentially enabling payment card fraud and non-compliance with PCI DSS and mobile data storage requirements.

Although the overall application risk is **MEDIUM**, this specific issue must be remediated before release because of its high severity and potential for sensitive financial data exposure.

Scope & Authorization

- **Target application:** re.pwnme
- **Package:** `re.pwnme`
- **Version:** 1.0
- **Target SDK:** 29
- **Size:** 3.5 MB
- **SHA-256:** `73799f94d9bce055509a14b1d33297aea81177d6a8ab8278dabfde3c17d8b41a`

Authorization note: This assessment was performed as part of a university graduation project on a lab-owned virtual Android device. No production systems, real users, or third-party infrastructure were involved.

OWASP Mobile Top 10 Classification

OWASP Mobile Top 10: M2 - Insecure Data Storage

This vulnerability falls under **M2 - Insecure Data Storage**. Logging a possible card number to Android logcat writes sensitive data into system logs, which are outside the app's sandbox and may be readable by other local processes or through ADB. The MASVS control **MASVS-STORAGE-2** specifically requires that no sensitive data be written to application logs.

Environment & Topology

COMPONENT	DETAIL
Tester host	ADB client host connected over TCP/IP
ADB transport	`10.0.2.5:5555`
Device	VirtualBox Android 9
Device model	`VirtualBox`
Device serial	`10.0.2.5:5555`
Installed app	`re.pwnme`
Native library	`/data/app/re.pwnme-0shNWy7MQKRBOJ7li5D7NA==/lib/x86_64/libnative-lib.so`

```
[ Tester host / ADB client ]
           |
           | TCP/IP :5555
           v
[ VirtualBox Android 9 device ]
           |
           | app under test
           v
[ re.pwnme v1.0 / libnative-lib.so ]
```

Methodology

1. **Static analysis** — The target APK was analyzed and the rule `LOG_CREDIT_CARD` matched, producing static evidence associated with the native library `libnative-lib.so`.
2. **Dynamic verification** — The application was launched using `adb shell monkey` to attempt to trigger the vulnerable logging path.
3. **Evidence capture** — Screenshots were captured before and after triggering. Logcat collection was attempted, but the filtering command failed and did not produce usable evidence.

Finding Detail

- **Rule ID:** `LOG_CREDIT_CARD`
- **Severity:** High
- **Confidence:** Medium
- **MASVS:** `MASVS-STORAGE-2`
- **Static recommendation:** Never log PANs.
- **Static evidence quoted verbatim:**

```
F/DEBUG   ( 2933):     #00 pc 0000000000044951  /data/app/re.pwnme-0shNWy7MQKRBOJ7li5D7NA==/lib/x86_64/
libnative-lib.so
```

The static evidence indicates a crash/debug log path in `libnative-lib.so`. The scanner identified this as a possible card number logging location.

Exploitation / Verification Steps

Step 1 — Capture baseline UI before triggering the vulnerable logging path

- Command: `screenshot`
- Result:

```
Screenshot saved as screenshot_01.png
```

Step 2 — Clear logcat to remove stale entries that could confuse evidence collection

- Command: `adb logcat -c`
- Result:

```
(exit 0, no output)
```

Step 3 — Launch re.pwnme to trigger the native code path that leads to card number logging/crash

- Command: `adb shell monkey -p re.pwnme -c android.intent.category.LAUNCHER 1`
- Result:

```
bash arg: -p
  bash arg: re.pwnme
  bash arg: -c
  bash arg: android.intent.category.LAUNCHER
  bash arg: 1
Events injected: 1
## Network stats: elapsed time=12ms (0ms mobile, 0ms wifi, 12ms not connected)
```

Step 4 — Wait for the app to reach the crash/logging state

- Command: `adb shell sleep 5`
- Result:

```
(exit 0, no output)
```

Step 5 — Dump logcat and filter for F/DEBUG, native library, and card-like numeric strings

- Command:

```
adb logcat -d -v long | grep -Ei 'F/DEBUG|libnative-lib|credit|card|[0-9]{4}[- ]?[0-9]{4}[- ]?[0-9]{4}
[- ]?[0-9]{4}' | head -n 100
```

- Result: **Failed**

```
Unrecognized Option E
Usage: logcat [options] [filterspecs]
...
```

The command failed because `logcat` received an unsupported `-E` option. The intended `grep -Ei` pipeline did not execute. No logcat evidence was captured.

Step 6 — Inspect the crash buffer for tombstone/F/DEBUG lines that may contain card data

- Command:

```
adb logcat -d -b crash -v long | grep -Ei 'F/DEBUG|backtrace|libnative-lib|credit|card|[0-9]{4}[- ]?[0-9]{4}[- ]?[0-9]{4}[- ]?[0-9]{4}' | head -n 100
```

- Result: **Failed**

```
Unrecognized Option E
Usage: logcat [options] [filterspecs]
...
```

The command failed with the same `Unrecognized Option E` error. No crash buffer evidence was captured.

Step 7 — Capture final device UI showing the result/proof after triggering the vulnerable logging

- Command: `screenshot`
- Result:

```
Screenshot saved as screenshot_07.png
```

Evidence

STEP	ARTIFACT	DESCRIPTION
Step 1	`screenshot_01.png`	Baseline UI before triggering vulnerable logging path
Step 3	Text output	Launch of `re.pwnme` via `adb shell monkey`; output shows `Events injected: 1`
Step 7	`screenshot_07.png`	Final device UI after triggering the vulnerable logging path

No logcat evidence was collected from Step 5 or Step 6 because both commands failed with `Unrecognized Option E`.

Remediation

1. **Remove all PAN logging from native code.** - In `re.pwnme`, inspect `libnative-lib.so` for logging calls such as `__android_log_print` or `__android_log_vprint`. - Remove or redact any statement that logs full card numbers.
2. **Never log PANs.** - If card data must appear in logs for debugging, log only the last four digits or a tokenized identifier. - Ensure the full PAN is never written to logcat, crash logs, or `F/DEBUG` output.
3. **Guard debug logging.** - Wrap debug-only logging with build-time flags such as `#ifdef DEBUG` or `BuildConfig.DEBUG`. - Strip debug logs from release builds.
4. **Implement centralized log sanitization.** - Add a logging wrapper that automatically masks PANs and other sensitive patterns before they reach logcat.
5. **Add CI static analysis rules.** - Fail builds when rules such as `LOG_CREDIT_CARD` match.

6. **Retest after remediation.** - Use a corrected logcat command, for example: `bash adb logcat -d -v long -e 'F/DEBUG|libnative-lib|credit|card|[0-9]{4}[- ]?[0-9]{4}[- ]?[0-9]{4}[- ]?[0-9]{4}'` - Confirm no card numbers appear in logcat or crash buffers.

Conclusion

The static scanner rated this specific finding as **High** severity. However, dynamic confirmation of exposed card number data was not achieved because the logcat filtering commands in Step 5 and Step 6 failed with an invalid option error. As a result, the live presence of a full PAN in logcat remains unverified in this test run.

Residual risk remains that `re.pwnme` may log possible payment card numbers from `libnative-lib.so` to Android system logs. If exploited or exposed, this could lead to payment card data disclosure and PCI DSS violations.

Follow-up actions: - Re-run log collection with corrected `logcat` syntax. - Inspect and fix the native logging path in `libnative-lib.so`. - Re-test the application to verify the issue is resolved. - Keep the overall app risk at **MEDIUM** until this High severity finding is remediated and confirmed closed.

Executed steps

Commands that ran and the output captured for each one.

1. Capture baseline UI before triggering the vulnerable logging path

COMMAND

```
screenshot
```

RESULT

```
Screenshot saved as screenshot_01.png
```

2. Clear logcat to remove stale entries that could confuse evidence collection

COMMAND

```
adb logcat -c
```

RESULT

```
(exit 0, no output)
```

3. Launch re.pwnme to trigger the native code path that leads to card number logging/crash

COMMAND

```
adb shell monkey -p re.pwnme -c android.intent.category.LAUNCHER 1
```

RESULT

```
bash arg: -p
  bash arg: re.pwnme
  bash arg: -c
  bash arg: android.intent.category.LAUNCHER
  bash arg: 1
Events injected: 1
## Network stats: elapsed time=12ms (0ms mobile, 0ms wifi, 12ms not connected)
```

4. Wait for the app to reach the crash/logging state

COMMAND

```
adb shell sleep 5
```

RESULT

```
(exit 0, no output)
```

5. Dump logcat and filter for F/DEBUG, native library, and card-like numeric strings

COMMAND

```
adb logcat -d -v long | grep -Ei 'F/DEBUG|libnative-lib|credit|card|[0-9]{4}[- ]?[0-9]{4}[- ]?
[0-9]{4}[- ]?[0-9]{4}' | head -n 100
```

RESULT

```
Unrecognized Option E
Usage: logcat [options] [filterspecs]
options include:
  -s              Set default filter to silent. Equivalent to filterspec '*:S'
  -f <file>, --file=<file>               Log to file. Default is stdout
  -r <kbytes>, --rotate-kbytes=<kbytes>
                  Rotate log every kbytes. Requires -f option
  -n <count>, --rotate-count=<count>
                  Sets max number of rotated logs to <count>, default 4
  --id=<id>       If the signature id for logging to file changes, then clear
                  the fileset and continue
  -v <format>, --format=<format>
                  Sets log print format verb and adverbs, where <format> is:
                    brief help long process raw tag thread threadtime time
                  and individually flagged modifying adverbs can be added:
                    color descriptive epoch monotonic printable uid
                    usec UTC year zone
                  Multiple -v parameters or comma separated list of format and
                  format modifiers are allowed.
  -D, --dividers  Print dividers between each log buffer
  -c, --clear     Clear (flush) the entire log and exit
                  if Log to File specified, clear fileset instead
  -d              Dump the log and then exit (don't block)
  -e <expr>, --regex=<expr>
                  Only print lines where the log message matches <expr>
                  where <expr> is a Perl-compatible regular expression
  -m <count>, --max-count=<count>
```

6. Inspect the crash buffer for tombstone/F/DEBUG lines that may contain card data

COMMAND

```
adb logcat -d -b crash -v long | grep -Ei 'F/DEBUG|backtrace|libnative-lib|credit|card|[0-9]{4}
[- ]?[0-9]{4}[- ]?[0-9]{4}[- ]?[0-9]{4}' | head -n 100
```

RESULT

```
Unrecognized Option E
Usage: logcat [options] [filterspecs]
options include:
  -s              Set default filter to silent. Equivalent to filterspec '*:S'
  -f <file>, --file=<file>               Log to file. Default is stdout
  -r <kbytes>, --rotate-kbytes=<kbytes>
                  Rotate log every kbytes. Requires -f option
  -n <count>, --rotate-count=<count>
                  Sets max number of rotated logs to <count>, default 4
  --id=<id>       If the signature id for logging to file changes, then clear
                  the fileset and continue
  -v <format>, --format=<format>
                  Sets log print format verb and adverbs, where <format> is:
                    brief help long process raw tag thread threadtime time
                  and individually flagged modifying adverbs can be added:
```

```
                  color descriptive epoch monotonic printable uid
                  usec UTC year zone
                Multiple -v parameters or comma separated list of format and
                format modifiers are allowed.
  -D, --dividers  Print dividers between each log buffer
  -c, --clear     Clear (flush) the entire log and exit
                  if Log to File specified, clear fileset instead
  -d              Dump the log and then exit (don't block)
  -e <expr>, --regex=<expr>
                  Only print lines where the log message matches <expr>
                  where <expr> is a Perl-compatible regular expression
  -m <count>, --max-count=<count>
```

7. Capture final device UI showing the result/proof after triggering the vulnerable logging

COMMAND

```
screenshot
```

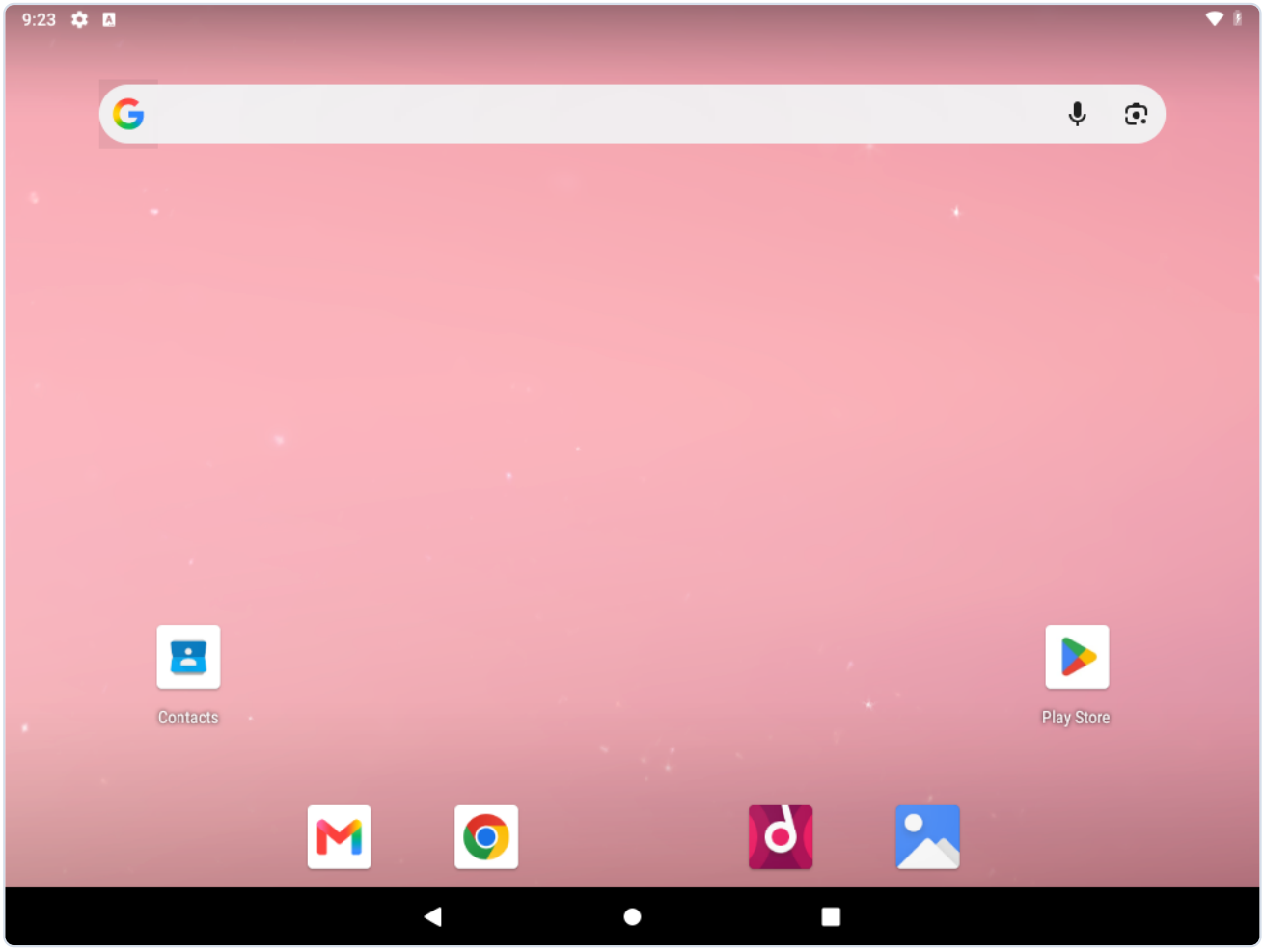
RESULT

```
Screenshot saved as screenshot_07.png
```

Captured evidence

Screenshots and command output saved during the run.

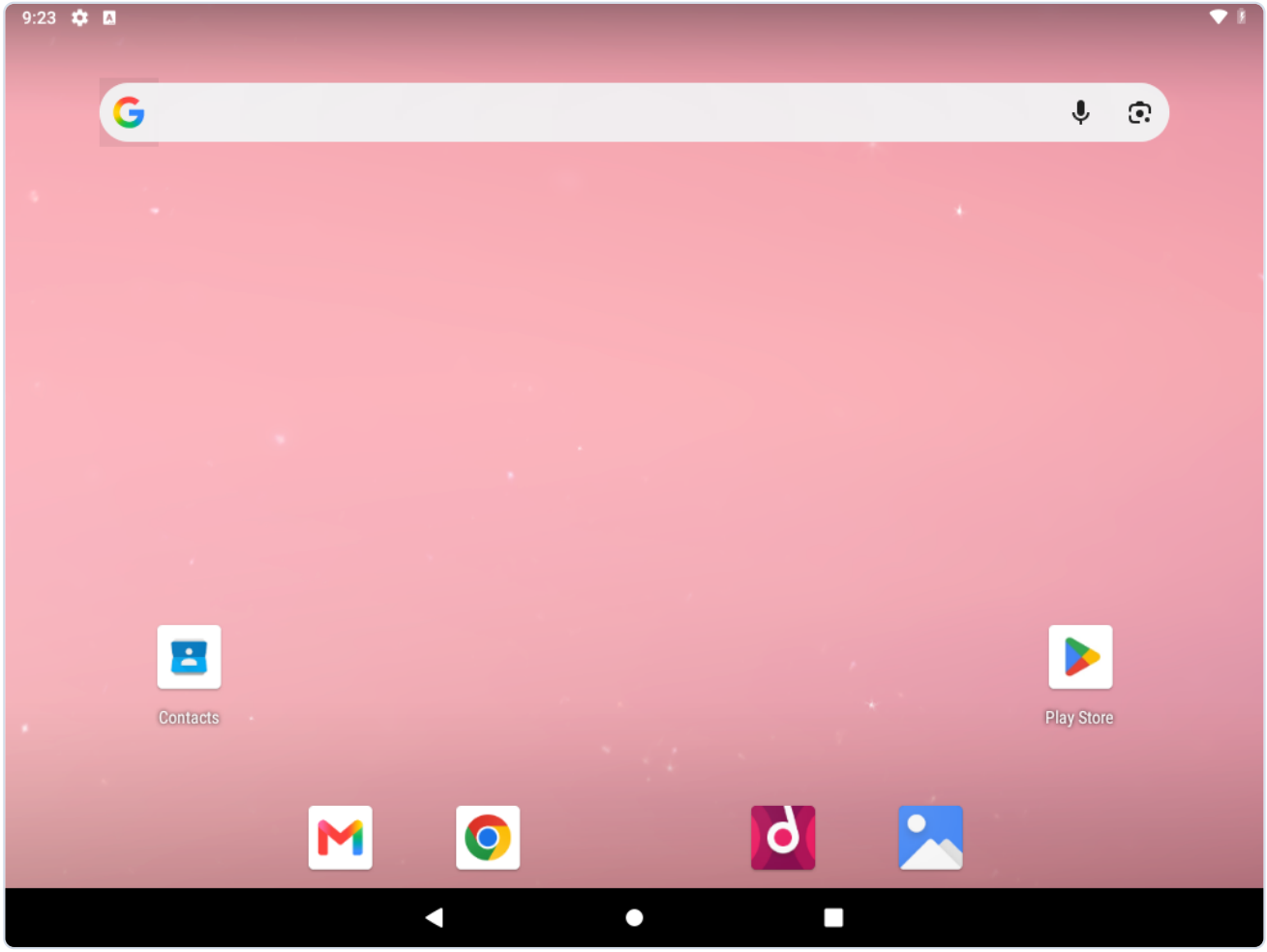
Screenshot — Capture baseline UI before triggering the vulnerable logging path



Command output — Launch re.pwnme to trigger the native code path that leads to card number logging/crash

```
bash arg: -p
  bash arg: re.pwnme
  bash arg: -c
  bash arg: android.intent.category.LAUNCHER
  bash arg: 1
Events injected: 1
## Network stats: elapsed time=12ms (0ms mobile, 0ms wifi, 12ms not connected)
```

Screenshot — Capture final device UI showing the result/proof after triggering the vulnerable logging



DISCLAIMER

This report is an automated assessment and provides an approximate security overview. It is not guaranteed to be 100% accurate. Some vulnerabilities may not have been detected.